

Data Center Failure Prediction System

Simple Guide: How to Run the Project and How it Works (PhD Methodology M1)

1. What is this Project? (Simple Explanation)

Imagine a massive computer server room (data center) with thousands of machines running 24/7. Occasionally, servers break down unexpectedly. However, **before a server crashes, it leaves early warning signs** — such as CPU usage spiking, memory filling up, or disk speeds slowing down.

This project builds an **Artificial Intelligence (AI) Early Warning System using XGBoost** that detects these warning signs **30 minutes BEFORE the server actually fails**. This gives system administrators time to safely move tasks to another computer before a crash occurs.

2. How to Run the Project (Step-by-Step)

You can run the entire project in **one single step** from your terminal:

Step 1: Open Command Prompt or Terminal.

Step 2: Navigate to your project folder:

```
cd "c:\Users\HINATA\Music\Bakya Akka"
```

Step 3: Run the python pipeline script (make sure to include **.py**):

```
python run_pipeline.py
```

Note: Running python run_pipeline without .py will cause an error! Always type python run_pipeline.py.

3. How the Pipeline Works (The 2 Modules)

The codebase is split into two logical modules to ensure clean data processing and **zero cheating (no data leakage)**:

Module & File	What it Does in Simple Terms
Module 1 Module1_Dataset_...ipynb	Simulates Server Telemetry: Generates 50,000 continuous time-series rows for 25 machines. Calculates 52 Rolling Features: Computes averages, spikes, and trends over past 25m, 75m, and 300m. Checks History Rules: Ensures feature math strictly uses <i>past</i> data (\$t' \le t\$).
Module 2 Module2_Forward_...ipynb	Creates Warning Labels: Flags \$y=1\$ if a server will fail in the next 30 minutes. Chronological Split: Divides data strictly by time (70% Train, 15% Val, 15% Test). 5-Point Audit: Programmatically confirms 100% zero data leakage. Trains AI Models: Trains XGBoost, Random Forest, and SVM classifiers.

4. Understanding the Results

When you run python run_pipeline.py, it trains the models and tests them on unseen future data. Here are the final benchmark results:

Model	Threshold	Precision	Recall	F1-Score	PR-AUC	ROC-AUC
Proposed XGBoost	0.860	65.8%	100.0%	79.3%	0.854	0.999
Random Forest Baseline	0.470	55.2%	100.0%	71.1%	0.796	0.999
SVM Baseline	0.430	69.8%	77.1%	73.3%	0.823	0.999

Key Findings:

- **Proposed XGBoost AI** achieves the best overall performance with an F1-Score of **79.3%** and PR-AUC of **0.854**.
- **100% Recall:** The system caught *every single failure precursor event* on test data without missing any!
- **Advance Warning Time:** Provides an average lead time of **30 minutes** before the server crashes.

5. Where Outputs Are Saved

All generated output files are stored cleanly in two subfolders:

- module1_outputs/: Raw telemetry parquet file & engineered feature parquet file.
- module2_outputs/:
 - xgboost_m1_precursor_model.json (Saved trained AI model)
 - m1_benchmark_results.csv (CSV table of all metrics)
 - m1_performance_and_feature_importance.png (High-res chart of PR Curves & Top 15 Features)
 - m1_lead_time_distribution.png (Chart showing 30-min lead time distribution)